

OceanData

Democratizing Oceanographic Insights & Real-Time Analytics

FULL CODEBASE EXPLANATION & TECHNICAL ARCHITECTURE STUDY GUIDE

Prepared for: Candidate Technical Interview

Codebase Scope: VishwaPratap777/Oceano (Full-Stack SPA)

Document Class: Technical Walkthrough & Prep Material

Date: July 01, 2026

1. System Overview & Architecture

OceanData is a full-stack, single-page web application designed to democratize access to oceanographic datasets and real-time marine metrics (such as Sea Surface Temperature (SST), wave heights, currents, wind speeds, and humidity). The system integrates three key layers: a modern React client with cinematic, fluid animations; an Express backend proxy server; and a multi-tiered AI assistant capable of processing real-time metrics and providing context-aware scientific explanations.

Key Interview Themes to Emphasize:

- **Production-Grade Fault Tolerance:** Highlight how the backend operates seamlessly even without API keys. The StormGlass service falls back to structured mock data, while the chatbot service implements a 3-tier cascade: Gemini 2.5 Flash -> Groq Llama 3.1 -> Local Keyword-Based Response Engine.
- **State-of-the-Art UX:** Focus on the aesthetic micro-interactions. The interface uses customized GSAP route transitions (staggered staircase slide-downs) overlaid with dynamic SVG water displacement filters, alongside custom canvas-based pointer ripple trails and Lenis smooth scrolling.
- **API Optimization:** Describe the in-memory coordinate caching system implemented in the weather API service, which uses a 1-hour Time-To-Live (TTL) to enforce rate limits and minimize external API network overhead.

Component Layer	Technologies & Libraries	Architectural Responsibility
Frontend Core	React, TypeScript, Vite, React-Router-DOM, TailwindCSS, @tanstack/react-query	Single-page application layout, optimized queries, cached client hooks, responsive grid structures.
Animations	GSAP (GreenSock), Framer Motion, Lenis Smooth Scroll, Canvas API	Cinematic rendering: staggered column stair transitions, water ripple SVG filters, active mouse particle trail, smooth scroll control.
Backend API	Node.js, Express.js, Axios, Dotenv, CORS	Express proxy routing, in-memory weather metric caching, environment loader, structured global exception middleware.
AI Services	@google/genai, groq-sdk, Llama-3.1-8b-instant, Gemini-2.5-flash	Context-aware system prompt composition incorporating live location weather details. Layered fallback pipeline.

2. Backend Architecture & File Walkthrough

The backend server is structured inside the server/ directory as a modular Node.js application leveraging ES Modules ("type": "module"). This section breaks down every backend file, explaining its code layout and how to present it in an interview.

Server Entry Point — server/src/index.js

Core Responsibility: Configures the main Express application, sets up global middlewares, registers core routes, serves production static assets, and boots the HTTP server.

Key Operations & Code Flow: Initializes Express and hooks up cors() and express.json() middlewares. Registers public endpoints (/api/ocean and /api/chat). Implements a health check route (/api/health) verifying the configuration of external APIs. For production deployment, it hosts the static compiled SPA assets out of ../../dist. Includes a catch-all route (*) forwarding non-API requests to the client-side SPA router (index.html).

■ **Interview Discussion Point:** Explain that in production, the node server hosts the client React SPA. This resolves common cross-origin request barriers (CORS) and simplifies deployment under a single, unified domain. Mention how it maps SPA route paths back to index.html so client-side react routing does not break on reload.

Environment Configuration Manager — server/src/config/env.js

Core Responsibility: Loads global variables from the root .env file, parses port settings, and holds references to model/weather credentials.

Key Operations & Code Flow: Uses dotenv.config targeting the root directory. Packages configuration into a clean config object mapping variables like STORMGLASS_API_KEY, GEMINI_API_KEY, and GROQ_API_KEY. Enforces default geocoordinates (Mumbai Coast: 18.9219 N, 72.8222 E). Outputs non-fatal console warnings (console.warn) on boot if required credentials are missing.

■ **Interview Discussion Point:** Emphasize the design choice of non-fatal warnings instead of throwing. This ensures developers can boot the app and immediately work on local features or design components without having to configure API key strings first.

Global Exception Interceptor — server/src/middleware/errorHandler.js

Core Responsibility: Catches unhandled route exceptions globally, logs their occurrences to stdout, and formats consistent JSON error responses for the client.

Key Operations & Code Flow: An Express global handler matching the 4-argument signature: (err, req, res, next). Determines the HTTP status (defaults to 500) and formats a clean JSON payload `{ error, status }`. In development environments, it prints full error stack traces to console stdout for rapid debugging, while shielding production clients from sensitive server details.

■ **Interview Discussion Point:** Talk about centralized error handling as a software engineering best practice. By capturing errors in a single middleware, it avoids redundant try-catch logs and guarantees that the client always receives a structured, predictable error response.

Ocean Metrics Endpoint — `server/src/routes/ocean.js`

Core Responsibility: Exposes a HTTP GET route to request live meteorological data based on user coordinates.

Key Operations & Code Flow: Handles requests at GET /. Extracts lat and lng query parameters, parses them, and passes them to the StormGlass weather service. Includes a try-catch block specifically intercepting StormGlass HTTP exceptions. If the StormGlass API returns a 402 (Payment Required due to free-tier quota exhaustion), it maps this to a 429 status code with a user-friendly error explanation.

■ **Interview Discussion Point:** Describe how mapping raw API codes (e.g. 402 to 429) makes client error handling logical. In an interview, explain how mapping payment errors to standard API rate limiting ('quota exceeded') improves customer transparency and guides UX design.

AI Conversations Handler — `server/src/routes/chat.js`

Core Responsibility: Exposes a HTTP POST endpoint processing user chat prompts enriched with real-time geographical conditions.

Key Operations & Code Flow: Processes payloads at POST /. Validates required body structures (message, context coordinates, history). Before generating the response, it attempts to fetch real-time weather metrics matching the coordinates. Wraps this secondary fetch inside a nested try-catch block; if it fails (due to quota or timeout), it logs a warning and proceeds, calling the chat generator with null context rather than crashing the request.

■ **Interview Discussion Point:** Emphasize the design paradigm of graceful degradation. Real-time data enrichment is treated as an enhancement: if the weather API fails, the user is still able to speak to the AI model using fallback oceanography data.

StormGlass API Driver & Caching — `server/src/services/stormglass.js`

Core Responsibility: Communicates with the external StormGlass meteorological API, maps nested data models, and enforces coordinate-based cache limits.

Key Operations & Code Flow: Utilizes axios. Declares a Map acting as an in-memory cache with a 1-hour TTL. If a cache hit matches the requested geokey (formatted coordinate query), it serves data instantly. Queries coordinates for wave height, current speed, water temperature, wind speed, swell, and humidity. Iterates source indexes (NOAA, METEO, DWD, etc.) to extract raw variables via a pick() helper. Generates a complete mock object with normal coordinates and metrics if the StormGlass key is missing.

■ **Interview Discussion Point:** Highlight the cache design. Point out that caching coordinates prevents API spam, saving network roundtrips and protecting the application from hitting free tier limits. Discuss how mapping weather source arrays to a fallback hierarchy guarantees high data fidelity.

Gemini AI Service — `server/src/services/gemini.js`

Core Responsibility: Primary AI model orchestrator using the modern `@google/genai`` library, framing context-aware prompts with strict marine safety guardrails.

Key Operations & Code Flow: Calls GoogleGenAI and targets the gemini-2.5-flash model. Builds a system instruction prompt by wrapping real-time ocean metrics (waves, current, wind) into the system guidelines. Enforces action-oriented instructions: e.g., if wave height exceeds 1.5m, the model must emphasize caution and issue safety warnings. Maps user history variables (converting ``assistant`` roles to ``model``) and triggers a content generation block. If no Gemini key is set or the call fails, it automatically redirects the prompt to the Groq service as a fallback.

■ **Interview Discussion Point:** Discuss the power of dynamic system prompt injection. Explain how injecting metrics into the system instructions gives the LLM immediate context, turning a generic model into an expert local advisor that responds dynamically to changing sea states.

Groq Fallback & Keyword Response Engine — `server/src/services/groq.js`

Core Responsibility: Operates as the secondary LLM fallback (hosting Llama-3.1) and houses a local keyword-matching database for zero-connection scenarios.

Key Operations & Code Flow: Utilizes groq-sdk targeting llama-3.1-8b-instant with a defined system instruction. If Groq credentials are missing or the API returns a quota error, it catches the exception and passes the query to an offline `getFallbackResponse()` function. This offline engine checks keywords (e.g. salinity, currents, wave, temperature, argo) against a list of pre-compiled, highly educational responses on ocean science.

■ **Interview Discussion Point:** Use this file to show off your focus on system availability. In an interview, explain how a multi-layer cascade (Gemini -> Groq -> Local Keyword Matching) ensures the chat UI remains functional under all failure modes, even if both LLM providers are offline or rate-limited.

3. Frontend Architecture & File Walkthrough

The client application is built inside the `src/` directory using React with TypeScript, Vite, TailwindCSS, Framer Motion, and GSAP. It is architected for premium, visual, and highly responsive user interfaces. This section analyzes the custom frontend scripts.

App Shell & Main router — `src/App.tsx`

Core Responsibility: Registers layout wrappers, configures API query clients, maps page routes, and coordinates global scroll transitions.

Key Operations & Code Flow: Declares the `QueryClientProvider` and `BrowserRouter`. Sets up routes for Index page (handling `/`, `/about`, `/data`, `/visualization`), Chat, Login, and Learn. Implements a throttled scroll listener tracking window depth. If the scroll position is on the landing hero and less than `10vh`, it hides the animated ocean background to prevent visual clutter. As the user scrolls down, the wave layers fade in.

■ **Interview Discussion Point:** Point out the scroll-based conditional background rendering. This is a subtle optimization that keeps the page lightweight on initial load and keeps the hero section's visual text clean before the user begins to explore.

Single-Page Scroll Orchestrator — `src/pages/Index.tsx`

Core Responsibility: Fuses landing page components and maps routing paths to smooth page scrolls.

Key Operations & Code Flow: Aggregates the Hero, AboutSection, HomeSections, HowWeDolt, and SiteFooter. Listens to changes in the URL location. If a route matches `/about` or `/data`, it calculates the target element's position and scrolls the window smoothly (accounting for the floating navbar offset) rather than jumping abruptly.

■ **Interview Discussion Point:** Discuss this approach as an SEO and UX best practice: it keeps a clean multi-page URL architecture for indexing, but presents a seamless, unified single-page layout to the user without full page reloads.

Chat Client Interface — `src/pages/Chat.tsx`

Core Responsibility: Manages chat conversation states, custom prompt submission, and rendering of chat history.

Key Operations & Code Flow: Maintains a message log array and is-loading states. Connects to `sendChatMessage`. Implements auto-scroll to the bottom of the conversation when new replies arrive using a `ref`. Includes a spring-animated sliding history sidebar (Framer Motion) and custom CSS keyframes mapping a typing indicator and backdrop glow.

■ **Interview Discussion Point:** Highlight the UI performance choices: use of Framer Motion spring stiffness for layout shifts, and CSS hardware-accelerated animations to run rendering loops efficiently without bogging down the main JS thread.

GSAP Staircase Page Transition — `src/components/Stairs.tsx`

Core Responsibility: Renders cinematic entrance and exit animations using staggered columns and SVG turbulence filters.

Key Operations & Code Flow: Contains 5 vertical gradient columns. When navigating to a new route, GSAP triggers a timeline: staggers the columns down to block out the view, shifts routes behind the curtain, and slides the columns down out of the viewport. During initial load, it executes an SVG-filtered water ripple animation on the logo for 3 seconds before sliding open, and stores a flag in `sessionStorage` to skip it on subsequent views.

■ **Interview Discussion Point:** *This is a high-impact visual feature. Explain how you used GSAP stagger loops to animate individual bars sequentially and how the SVG displacement map filter creates a realistic watery effect. Talk about how coordinates are saved in `sessionStorage` to optimize subsequent navigation.*

GSAP Wave & Bubble Animation — `src/components/backgrounds/OceanBackground.tsx`

Core Responsibility: Builds a fluid background overlay containing looping SVG waves and random floating bubble streams.

Key Operations & Code Flow: Declares three layered wave SVGs at the bottom of the screen. GSAP translates these waves along X to -50% for a seamless horizontal loop, while simultaneously bobbing them vertically. Fires two vertical streams of bubble particles (randomizing scale, velocity, and drift) along the screen margins, and listens to tab visibility changes to pause loops and save CPU cycles.

■ **Interview Discussion Point:** *Mention the performance optimizations. Discuss how using 2 matching tiles moving from 0 to -50% creates a seamless loop. Explain how you integrated tab visibility listeners to stop GSAP tweens when the tab is hidden, preventing CPU drain.*

Canvas Mouse Ripple Trail — `src/components/effects/MouseRippleTrail.tsx`

Core Responsibility: Renders smooth, watery cursor trails on a transparent canvas overlay.

Key Operations & Code Flow: Creates a full-screen Canvas overlay. Throttles cursor movements to ~30fps to protect performance. Draws expanding radial gradients (cyan to transparent) on each mouse step, fading them out using a standard decay math. Uses `devicePixelRatio` matching to scale the canvas cleanly on high-DPI displays.

■ **Interview Discussion Point:** *Explain that drawing mouse trails directly in the DOM using divs causes huge layout overhead. By rendering directly onto a single, hardware-accelerated 2D canvas context and throttling listeners, we achieve smooth, high-fidelity trails with zero layout shifts.*

Cinematic Hero Showcase — `src/components/Hero.tsx`

Core Responsibility: Controls loop timers for video masks, manages ambient audio elements, and triggers intro transitions.

Key Operations & Code Flow: Contains a main background video and a header text mask that clips two identical videos. Uses a custom crossfade `timeupdate` handler that starts playing the second video 1.2s before the first video ends, swapping opacities to hide the loop loop gap. Manages ambient ocean sounds with volume ramps, and hooks into window visibility to pause play states.

■ **Interview Discussion Point:** *Explain the video loop issue. HTML5 videos pause for a split-second when looping, showing a black frame. By running two video elements side-by-side and performing a custom crossfade before the*

active video ends, we create a continuous, seamless loop. Explain how this is achieved through timeupdate listeners.

4. Deep Dive: Core Engineering & Optimization Strategies

In technical interviews, interviewers frequently dive deep into optimization, API choices, and advanced UI design. Be prepared to explain these three core strategies from the OceanData project:

A. Seamless Video Loop with Masked Typography

Standard HTML5 video elements suffer from loop latency—a brief pause or black flash occurs when the video resets. To achieve a premium, high-fidelity hero section, OceanData uses a dual-video crossfade system:

1. Two video elements (v0 and v1) are loaded with the same source.
2. An active variable tracks which video is visible, setting its opacity to 1 and the other to 0.
3. A timeupdate listener monitors the playing video. When the remaining play time is less than the crossfade duration (1.2 seconds), the hidden video starts playing from 0.01 seconds.
4. A GSAP/CSS transition swaps the opacities, creating a seamless visual crossfade. Once the transition is complete, the active variable shifts, and the process repeats. This completely bypasses loop stutter.

B. Smart Multi-Tiered AI Fallback Cascade

OceanData does not rely blindly on a single LLM API. To ensure high availability, the backend employs a robust three-tier cascade:

- **Tier 1 (Gemini 2.5 Flash):** The primary chatbot. It uses the modern `@google/genai` library and builds a location-aware system prompt using real-time StormGlass metrics. It applies business logic, like warning users if waves exceed 1.5m.
- **Tier 2 (Groq Llama-3.1):** If the Gemini key is missing or the request fails (e.g. rate limits), the router catches the exception and redirects the message and history to the Groq service, query-mapped to Llama-3.1.
- **Tier 3 (Local Keyword Engine):** If Groq is also down or unconfigured, the service falls back to a smart, local dictionary. It checks the user's message for keywords like *salinity*, *monsoon*, or *argo* and returns pre-written, highly detailed explanations. This ensures the chat interface always responds to the user.

C. Performance-First Animation Design

To achieve rich aesthetics without lagging the main browser thread, OceanData implements several key performance measures:

- **Canvas for Particle Effects:** The mouse ripple trail is rendered using a single, transparent 2D canvas overlay. Instead of spawning hundreds of DOM elements (which triggers constant layout thrashing), ripples are drawn as radial gradients and cleared on a lightweight animation loop.
- **GSAP Tween Pausing:** When the user switches tabs, both the ocean background waves and the smooth scroll library (Lenis) detect the page visibility change and immediately pause their animation loops. This saves significant CPU/GPU resources and prevents battery drain.

5. Common Interview Q&A

This section lists questions an interviewer might ask about this codebase, along with high-scoring technical responses.

Q: Why did you build an Express backend proxy instead of calling the weather APIs directly from React?

A: Calling external APIs (like StormGlass) directly from React has two major flaws: first, it exposes sensitive API keys to the client console, which is a severe security risk; second, it triggers Cross-Origin Resource Sharing (CORS) blocks if the third-party server doesn't allow direct client requests. Setting up an Express proxy server solves both: keys remain secure in the backend's environment variables, and the client makes clean, local-origin calls to /api. It also lets us implement in-memory caching to save API costs.

Q: How does the caching mechanism in the StormGlass service work, and how does it handle expired cache values?

A: The StormGlass service implements an in-memory Cache using a standard JavaScript Map. The cache key is derived from coordinates rounded to two decimal places, creating a grid bounding box of about 1.1km. When a request comes in, we check if the key exists and if the current timestamp is less than the cache expiration time (1-hour TTL). If both conditions are met, we serve the cached weather data instantly. If the cache has expired, the entry is deleted and a fresh request is sent to the StormGlass API. The new data is then stored in the cache map with a new expiration timestamp.

Q: How is the AI chat page designed to handle deep history contexts, and how does it prevent context-window bloat?

A: In Chat.tsx, the client-side state maintains the full list of messages. However, when sending a request to the backend, we filter out the initial greeting and map the message history into an array of roles and content. In the backend, the Groq service restricts history context by slicing only the last 10 messages (history.slice(-10)) before forwarding them to Llama-3.1. This prevents context-window bloat and controls token usage. The Gemini service formats the last few conversation steps directly into its structured contents array, ensuring the model retains relevant context without incurring massive billing costs.

Q: What performance optimizations did you make to the custom GSAP animations in the background wave layers?

A: First, we used CSS 3D transforms (force3D: true) on the wave containers to force the browser to upload the wave layers to the GPU, which ensures hardware-accelerated animations. Second, instead of rendering a continuous wave path dynamically, we designed two identical SVG wave path tiles. We place them side-by-side in a container, translate the container horizontally to exactly -50%, and loop. This creates a seamless wave animation with minimal rendering cost. Finally, we pause these loops when the page is hidden by listening to the tab visibility API, saving processor cycles.

Q: How is the custom page transition coordinated between React Router and the GSAP animators?

A: We wrap our page routes inside a PageTransitionWrapper component, which defaults to the Stairs animator. The Stairs component uses React's `useLocation` hook to detect route changes. When the path changes, the component's `useEffect` hook triggers a GSAP timeline: it blocks user input, staggers 5 vertical columns down to cover the screen, triggers custom events (`stairs:start`, `stairs:mid`, and `stairs:complete`), and then glides the columns down to reveal the new page content. It also pauses our smooth-scrolling library (Lenis) during the transition to prevent visual stutter.